

WHEN TO COMPILE A COMPUTER-USE AGENT? MEASURING PAYBACK AND MAKING COMPILATION DECISIONS FOR TOKEN EFFICIENCY

Anonymous authors

Paper under double-blind review

ABSTRACT

Compiling GUI procedures that agents execute repeatedly into programs can reduce their token costs. However, deciding when to compile poses two challenges. First, the compilation cost is unknown before an attempt, which may fail after consuming tokens. Second, future use is unknown because tasks may stop repeating or GUI changes may break the program. To address these challenges, we propose PACE (Payback-Aware Compilation from Experience), a system with a measurement protocol and an online compilation algorithm. The measurement protocol measures compilation costs and per-use savings to estimate how many uses a program needs to pay back its compilation cost. Using these measurements, the online algorithm estimates whether future savings justify compilation based on past task arrivals and compilation outcomes. It checks execution and compilation charges against a cumulative budget before allowing either action. Under stated action-cost assumptions, total cost after each arrival is at most $1 + \epsilon$ times the cost of running every task with the agent. Based on measured costs, verified programs need an estimated 1–14 uses to recover the cost of a successful compilation. In experiments using recorded task arrivals, PACE reduces token costs by 21.9% compared with ReAct, 26.9% with AutoRPA, and 21.6% with ToolPro on average ($\epsilon = 0.25$).

1 INTRODUCTION

Large language models have advanced rapidly. This progress enables computer-use agents. A computer-use agent interacts with applications through graphical user interfaces (GUIs). At each step, the agent reads the GUI as both a screenshot and a structured text tree, e.g., an accessibility or DOM tree (Hong et al., 2024; Zhang et al., 2025). It then runs a ReAct-style (Yao et al., 2023) agent loop, i.e., it calls the model to generate actions and executes them on the interface. Task success rates keep rising on benchmarks of web, desktop, and mobile tasks (Xie et al., 2024; Rawles et al., 2025). However, computer-use agents consume more tokens than general agents because of the screenshot inputs, making them more expensive to run. Token efficiency, i.e., reducing the token cost, has become a first-order concern.

To improve token efficiency, prior systems reuse earlier experience. When an agent solves similar tasks repeatedly, starting from scratch requires it to rediscover the same procedure each time. To reduce this cost, prior systems store earlier experience as text (e.g., memory items or skills) (Wang et al., 2025b; Zheng et al., 2024) or *compiles* it into parameterized programs by generating code (Wang et al., 2025a; Li, 2026; Chen et al., 2026a; El Yadouni & Li, 2026). The program gets the inputs from a model call to extract them and executes without further model calls. We focus on the program method, which has the highest token efficiency.

However, a low execution cost does not mean that compilation saves tokens over the full task stream. According to prior systems’ audits, the *payback count*, i.e., the number of uses whose token savings can cover the compilation cost, ranges from 4 (Chen et al., 2026a) to 17 (Trooskens et al., 2026). Most systems report no payback count (El Yadouni & Li, 2026; Xu et al., 2026). Moreover, these audits measure the costs of different system parts, making the numbers incomparable. We therefore ask two questions. *How can we systematically measure the payback count? When is it worth compiling a program instead of continuing to use the agent?*

Measuring the payback count and deciding when to compile face two challenges. **Uncertain compilation cost.** The system compiles into a program with multiple attempts. Each failed attempt requires repairing the program, which consumes tokens. Particularly, if the system fails to produce the program and falls back to the agent, the compilation cost is wasted. **Unknown future use.** The system does not know how many later tasks can use the program. Similar tasks may stop arriving before they pay back the spend. The GUI changes that make the program not work, i.e., *GUI drift*, can also break a deployed program and requires a compilation from scratch. Therefore, compiling early can waste tokens if tasks do not repeat, while waiting can lose savings on tasks that repeat often. An online algorithm is required to make this choice using past observations.

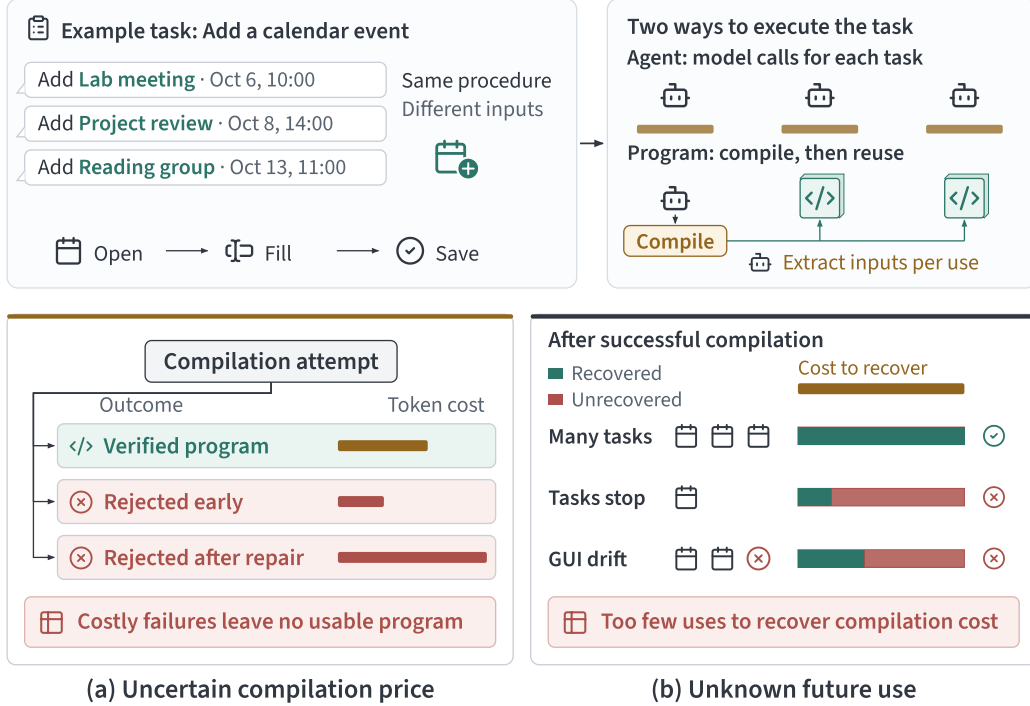


Figure 1: Challenges in deciding when to compile GUI procedures. Top: an example task and its agent and program execution paths. In (a), red cost bars mark rejected compilation attempts. In (b), red bar segments show compilation costs left unrecovered when task arrivals stop or GUI drift ends reuse.

We propose *PACE* (Payback-Aware Compilation from Experience), a system with a measurement protocol and an online compilation algorithm. *PACE* uses experience in two ways: completed agent traces provide the procedure to compile, while past task arrivals and compilation outcomes inform the decision to compile. The measurement separates the cost of obtaining a program from the saving after that program exists. The serving comparison uses matched task inputs and includes agent fallback after a detected program failure. The protocol compiles recorded agent traces, selects programs through a validation gate, and charges the model calls on both successful and unsuccessful compilation paths. Deployment runs measure extraction cost and program failures. Additional compilations test whether the initial outcome repeats, and agent replays on the program’s task inputs check the serving comparison. Across seven task families and three models, the measurements show per-use savings alongside differences in compilation cost and outcome.

PACE uses these measurements to decide when to compile under a cumulative cost constraint. A cost-aware compilation proposal compares estimated future savings with the estimated compilation cost of obtaining a verified program, including failed attempts. A separate budget check admits an action only when its possible cost fits the allowance determined by the task arrivals already observed. Total cost includes serving, routing, and every successful or failed compilation. Under the stated cost model, *PACE* spends at most $1 + \epsilon$ times the reference cost of running every task with the agent

after each arrival. The margin ϵ limits the extra spend even when future-use estimates are wrong or compilation keeps failing. We evaluate the resulting savings and tradeoffs with measured cost profiles, synthetic arrivals, and arrival orders from public logs.

Our contributions are:

- We propose a measurement protocol for the cost and saving of compiling computer-use tasks. We report serving costs, compilation outcomes, repair costs, and payback counts under a shared accounting scheme, with repeated compilations and matched task-input replays.
- We propose an online compilation algorithm that uses a cost budget to decide whether to compile. PACE makes cost-aware proposals from past experience and admits charges through a cumulative budget. We prove a cumulative bound under action-cost assumptions.
- We implement and evaluate across cost and task repetition settings. We retain the measurement records, policy implementations, and paired experimental outcomes. With $\epsilon = 0.25$, PACE reduces token costs by 21.9% relative to ReAct, 26.9% relative to the AutoRPA adaptation, and 21.6% relative to the ToolPro adaptation on average across the recorded-arrival experiments. Ablations separate the effects of the savings test, cost budget, and failed-attempt cost.

2 PROBLEM FORMULATION

We study when to compile GUI procedures that agents execute repeatedly while limiting cumulative cost relative to continued agent service. Tasks in one family share a GUI procedure but differ in their inputs, such as event titles and dates.

We distinguish two ways to serve a task. In *agent execution*, the model selects GUI actions step by step, with calibrated mean cost c . In *program execution*, a model call extracts the task inputs, or *binding*, and a parameterized program then executes the shared procedure without further model calls. Its model cost c^{prog} is the extraction cost. We use weighted tokens: one unit costs as much as one uncached input token for that model, with cached-input and output tokens weighted by their relative prices. Appendix A.2 gives the accounting formula and calibrations.

A compilation attempt uses k completed agent traces. The verifier replays the source tasks with model-extracted inputs and checks whether the program completes them. We assume access to task resets and success checks for these completed tasks. The attempt charge includes extraction for verification and any bounded repair. Let p be its verification probability, C the conditional mean successful compilation cost, and C^{fail} the conditional mean failed compilation cost. A failed attempt pays its cost without producing a program. In the online model, these calibrated costs become fixed charges for the corresponding outcomes. Historical means alone do not bound future real bills.

Without GUI drift, a program run fails with probability q and requires one agent fallback of cost c . GUI drift breaks a program with probability h per use, requiring fallback and invalidation. Ordinary failure occurs with probability q conditional on no break. The cost model assumes detected failures and reports task success separately. Routing costs $\tau_0 + m\ell_t$ when ℓ_t programs are listed, where τ_0 is the empty-manifest charge and m is the per-entry charge.

Objective. We compare PACE with serving every arrival through agent execution, without compiling or reusing programs. For arrival t , define this agent execution reference charge as $a_t = \tau_0 + c$ using that family’s supplied cost, and let

$$A_t = \sum_{i=1}^t a_i. \quad (1)$$

This reference includes program-served arrivals. It is a model-based comparison, not an extra agent execution observed on each task. Let K_t^π include policy π ’s routing, serving, fallback, and all compilation charges through arrival t , including compilation after the final service. For an unknown stream length n , we seek

$$\min_{\pi \in \Pi_{\text{online}}} \mathbb{E}[K_n^\pi] \quad \text{subject to} \quad K_t^\pi \leq (1 + \epsilon)A_t, \quad t = 1, \dots, n. \quad (2)$$

The constraint holds for every history allowed by the cost model, while the expectation uses the arrival and outcome model evaluated in the experiments. The class Π_{online} contains policies that

serve each arrival through an agent or an available program, using only the current task, supplied parameters, and past observations. We supply family identities, cost profiles, routing charges, and q, h . The policy estimates future task repetitions and verification success online. Artifact lookup and manifest removal incur no token charge. Savings can fund compilation across families, but unobserved future arrivals add no budget. The margin is $\epsilon = 0.25$ in the main evaluation. This objective constrains cost, without imposing a task-success guarantee.

3 MEASUREMENT

We measure both the saving after a program exists and the cost of obtaining it. The protocol charges successful and failed compilation attempts, evaluates verified programs on fresh bindings, and checks the serving comparison through matched agent replays. These measurements provide the online cost profiles and show why low program-use cost alone does not establish payback.

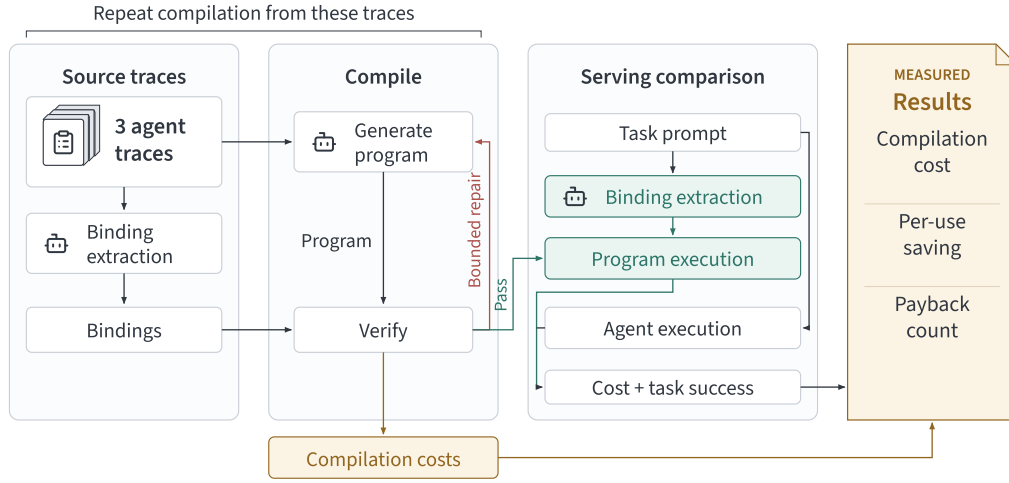


Figure 2: PACE’s measurement protocol. The source traces supply both the compilation input and the bindings extracted by a model call for verification. Fresh task prompts are used to compare program and agent execution. The result panel combines serving measurements with the costs of successful and failed compilation attempts.

Protocol. We test seven families: four AndroidWorld tasks (Rawles et al., 2025), two custom LibreOffice tasks in an OSWorld-based container (Xie et al., 2024), and a comment-posting task on WebArena’s Reddit application (Zhou et al., 2024). The models are GLM 5.3 Flash, DeepSeek Flash, and Qwen 3.8 Flash, with exact recorded identifiers in the Appendix. Following AutoRPA’s translation and construction stages (Chen et al., 2026a), we compile $k = 3$ completed agent traces and permit bounded repair. For verification, the model extracts bindings from the three source task prompts. We cache these bindings within the attempt, restore each task’s initial state, and require all three tasks to succeed before deployment. This check measures reproduction of the construction tasks. Thirty fresh deployment tasks provide a separate evaluation, including model-based binding extraction. We subtract the measured no-task harness cost from agent runs, and retain every recorded compilation and repair charge. The reported costs exclude calls without retained records. Some earlier Android cache counts require imputation. Appendix A specifies the verification procedure, calibrations, and retained-record coverage.

3.1 SERVING COST

For a verified program, the no-drift expected saving is

$$s = (1 - q)c - c^{\text{prog}}, \quad \text{saving share} = s/c. \quad (3)$$

The saving calculation includes extraction cost and one mean-cost fallback per failed program run. Fifteen initial programs pass verification. Their extraction costs range from 179 to 836 weighted

tokens and their estimated saving shares from 0.794 to nearly 1. Appendix A.4 reports the profiles used in the experiments.

We replay the agent on the thirty deployment bindings of each of the nine verified Android programs, giving 270 matched-input agent runs (Table 1). Their saving shares remain 0.794–0.999 despite differences in agent and program success. For example, GLM Calendar and OsmAnd programs each pass all thirty bindings, while the replayed agent passes fourteen and twelve. The replay uses a later emulator profile and estimates fallback cost as qc ; it does not measure an agent resumed after each failed program run.

Table 1: Agent replay costs and success counts on thirty matched deployment bindings per populated entry. c is the mean baseline-subtracted cost in K weighted tokens, with $K=10^3$. Saving share uses the mean-cost fallback estimate qc . Blue, amber, and green identify GLM, DeepSeek, and Qwen, respectively. Appendix A.8 reports the confidence intervals.

Family	Mean c (K tokens)			Agent successes (of 30)			Program successes (of 30)			Saving share (fraction)		
Contacts	67.1	67.3	42.2	30/30	27/30	30/30	30/30	29/30	29/30	0.997	0.960	0.954
Markor delete	35.4	22.3	19.3	30/30	23/30	30/30	24/30	29/30	30/30	0.794	0.949	0.980
Calendar	535.3	–	315.6	14/30	–	15/30	30/30	–	30/30	0.999	–	0.997
OsmAnd marker	258.9	–	–	12/30	–	–	30/30	–	–	0.998	–	–

3.2 COMPILATION COST

Table 2 reports the initial attempt charges and validation outcomes. Using the recorded successful charge as C , the marginal payback is C/s . Adding k mean-cost source runs gives $(C + kc)/s$. These ratios exclude earlier failed attempts, extra runs needed to acquire traces, and replacement after drift. We report unrounded cost ratios; a whole-use threshold for covering the recorded charge in expectation takes the ceiling. The fifteen verified programs come from twenty cells with compilation-stage records. Qwen’s web request is refused, and its Calc record contains only partial spend.

Table 2: Initial compilation costs, validation counts, and conditional payback. The two payback columns report C/s and $(C + kc)/s$, with $k = 3$ agent runs here. Payback excludes earlier failed attempts and future replacement. Blue, amber, and green identify GLM, DeepSeek, and Qwen, respectively. Qwen Calc gives partial spend; dashes mark unavailable values.

Family	Attempt cost (K tokens)			Initial pass (of 5)			Final pass (of 5)			Compilation-only payback (uses)			Trace-inclusive payback (uses)		
Contacts	41.0	144	137	5/5	5/5	5/5	5/5	5/5	5/5	0.416	2.30	3.66	3.42	5.43	6.81
Markor delete	39.0	68.5	208	5/5	5/5	5/5	4/5	5/5	5/5	1.46	6.85	9.83	5.23	10.1	12.9
Calendar	916	1100	1540	0/5	0/5	0/5	4/5	0/5	4/5	2.93	–	11.5	5.93	–	14.5
OsmAnd marker	967	2500	1590	3/5	0/5	3/5	5/5	0/5	1/5	3.46	–	–	6.47	–	–
Calc table	481	4370	848	0/5	0/5	–	5/5	0/5	–	0.954	–	–	3.96	–	–
Writer memo	254	586	486	0/5	0/5	5/5	5/5	5/5	5/5	3.65	11.7	13.4	6.68	14.7	16.5
Reddit comment	253	332	–	0/5	2/5	–	5/5	5/5	–	2.05	8.44	–	5.07	11.5	–

DeepSeek’s failed attempts have median cost 2500K weighted tokens, compared with 238K for successful attempts, a 10.5-fold difference across families. Its Calc attempt costs 4370K and passes none of five bindings after repair. This comparison does not identify the within-family cost effect of failure. Conditional marginal payback ranges from 0.416 to 13.4 uses across verified programs, so cheap serving can still require substantial reuse.

Twenty-two additional attempts produce fourteen verified programs, five verification failures, and three provider interruptions. Qwen Calendar, for example, has both a successful and a failed compilation attempt under the same verifier. Appendix A.5 reports these attempts and program rechecks. The small, partly changed-environment samples do not establish independent retries. The GUI perturbation probe also finds silent failures, limiting the detected-failure assumption used in the online model.

4 ONLINE ALGORITHM

PACE’s online compilation algorithm uses observed task arrivals and compilation outcomes to make cost-aware compilation proposals, with a separate cumulative budget to admit charges. The proposal can be wrong about future arrivals or verification, while the budget uses only arrivals already observed and costs already charged. The algorithm serves the current task before considering compilation for later tasks.

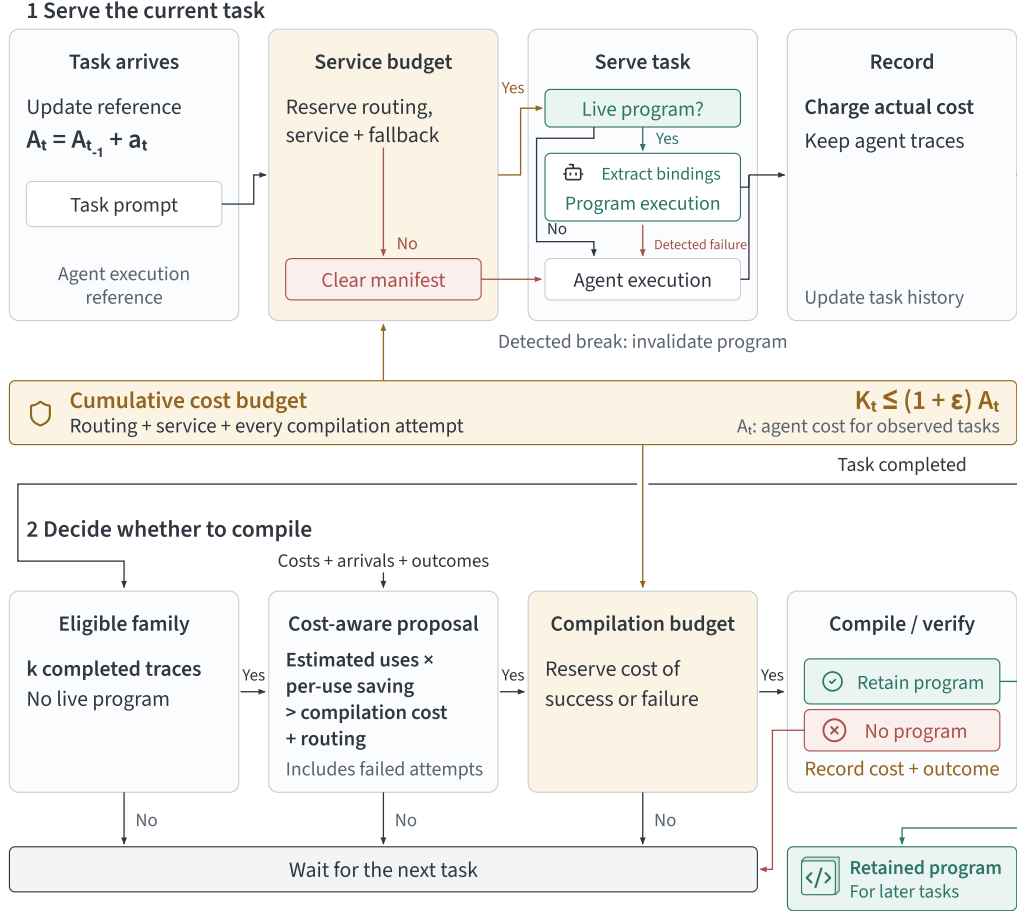


Figure 3: PACE’s online compilation algorithm. After serving a task, PACE checks trace availability, estimated savings, and the cost budget before compiling. A failed check leads to waiting, while successful verification adds a program for later tasks. The two budget checks use the same cumulative cost budget.

4.1 COST-AWARE COMPILATION PROPOSAL

A family becomes eligible after k completed agent runs, including fallbacks. Let N be its observed arrival count and d its age in total stream arrivals. After j compilation attempts and g successful verifications, define

$$\hat{H} = \min\left(\frac{Nd}{d+5}, \frac{1}{h}\right), \quad \hat{p} = \frac{g+1}{j+2}, \quad \hat{B} = C + \left(\frac{1}{\hat{p}} - 1\right) C^{\text{fail}}. \quad (4)$$

We use $1/0 = \infty$. The smoothed count estimates future uses, capped by the supplied mean program lifetime. It is a heuristic estimate from observed arrivals, without a fitted prediction model. The estimated compilation cost $\hat{B}$ includes failed attempts before obtaining a verified program, under the

independent-retry interpretation derived in Appendix B.1. Neither interpretation is required for the cost guarantee.

With drift-adjusted saving $s_h = c - c^{\text{prog}} - [h + (1 - h)q]c$, an eligible family without a live program proposes compilation when

$$s_h > 0 \quad \text{and} \quad \hat{H}s_h > \hat{B} + m \min(T, d). \quad (5)$$

The final term allows for one manifest entry during its estimated residence. Entries expire after T idle arrivals, while retained programs can be relisted until a detected break invalidates them. The proposal accounts for the two sources of uncertain payback separately. Low verification probability increases the estimated number of paid failures before obtaining a program, whereas GUI drift reduces the estimated number of uses after verification. An expensive failed attempt can therefore make compilation unattractive even when successful construction is cheap. The following budget checks control the cost incurred when these estimates are wrong. Appendix C.6 compares alternative proposals with the same budget. Their similar costs do not identify the exact smoothing formula as a separate contribution.

4.2 BUDGET CHECKS

Before routing, PACE expires idle entries and tentatively relists any retained live program for the arriving family. The manifest exposes programs to the model for routing, and its cost is paid on every arrival, including tasks served by the agent. Retaining an artifact incurs no token cost, but leaving it listed can keep incurring routing cost. For the resulting manifest size ℓ_t , it reserves

$$U_t^{\text{serv}} = \tau_0 + m\ell_t + \begin{cases} c, & \text{agent service,} \\ c^{\text{prog}} + c, & \text{program service.} \end{cases} \quad (6)$$

The program reservation includes a possible fallback. Service proceeds only if $K_{t-1} + U_t^{\text{serv}} \leq (1 + \epsilon)A_t$. Otherwise PACE clears the manifest before routing and uses the agent, whose charge is at most a_t . Clearing preserves retained artifacts and leaves no mandatory future charge.

Let $\tilde{K}_t$ be the cumulative charge after service. A proposed compilation proceeds only if

$$\tilde{K}_t + \max(C, C^{\text{fail}}) \leq (1 + \epsilon)A_t. \quad (7)$$

PACE then adds the realized compilation charge to its cumulative cost, including failed attempts and any compilation after the last service. Savings can fund compilation across families, but estimated future uses add no budget. Appendix B gives the complete execution order.

Budget example. Consider a proposed compilation with success and failure charges of 60 and 120 weighted tokens. If the cumulative reference and cost already charged are both 300, a margin of 0.25 leaves 75 tokens available. PACE defers compilation because a failure would exceed the 375 limit, even if the estimated saving justifies its estimated compilation cost. After two further agent-served arrivals costing 100 each, the reference and charged cost are both 500. The available 125 tokens now cover either outcome. Only observed arrivals increase this allowance.

4.3 COST GUARANTEE

Theorem 4.1 (Cost bound relative to agent execution). *Let the reference increments a_t be nonnegative. Suppose every proposed action has a known finite charge bound, and clearing the manifest before routing costs no tokens, leaves no mandatory future charge, and permits default service at cost at most a_t . For any $\epsilon \geq 0$, PACE with exact arithmetic and valid reservations satisfies*

$$K_t \leq (1 + \epsilon)A_t \quad \text{for every completed-arrival prefix } t \quad (8)$$

on every history allowed by these charge bounds.

Proof. The bound is preserved in two steps. An admitted service costs no more than its reservation. If admission fails, default service adds at most a_t , so

$$K_{t-1} + a_t \leq (1 + \epsilon)A_{t-1} + a_t \leq (1 + \epsilon)A_t.$$

The compilation check then preserves the same bound for either outcome. Starting from $K_0 = A_0 = 0$ establishes the result by induction. $\square$

The fixed-charge model satisfies the assumptions with $a_t = \tau_0 + c$. The implementation permits tolerance $10^{-8} \max\{1, (1 + \epsilon)A_t\}$. The bound is global over the stream and concerns cost, not task errors or per-family spending. For variable real bills, reservations must be enforceable and default service affordable. Historical means alone do not supply these conditions. PACE is a feasible algorithm for Equation 2, without a claim of optimizing it.

5 ONLINE EVALUATION

We evaluate the savings available under PACE’s cumulative budget, its comparison with published decision rules, and the contribution of each component. The study uses measured cost profiles on synthetic streams and recorded arrival orders.

Setup. We run two grids of 300 conditions, each crossing three models, four verification settings, five arrival patterns, and five cost or environment settings, with eight paired repetitions per condition. The second grid changes seeds and parameters after the first study. We also use Sepsis and BPI 2019 process logs (Mannhardt, 2016; van Dongen, 2019) and two Wikipedia edit streams, giving twelve combinations of models and logs with ten paired repetitions. The logs supply task repetitions and their order, not live GUI tasks. We shuffle measured cost profiles onto families and assign verification probabilities independently of initial compilation outcomes. Unobserved successful or failed costs require substitutions, whose effect we test below. The policies receive family labels and failure parameters, including detected-break status. Appendix C specifies the grids, mappings, substitutions, and shared outcome sequences.

We compare PACE with ReAct (Yao et al., 2023) and decision-rule adaptations of AutoRPA (Chen et al., 2026a) and ToolPro (Liu et al., 2026) under the same compiler and cost model. ReAct serves every arrival with the agent. AutoRPA makes one compilation attempt when enough source traces are available and does not retry a rejected attempt. ToolPro compares the compilation cost with the saving available from the current task, without projecting later uses. These adaptations isolate when to invoke the shared compiler. All policies pay the same routing, extraction, fallback, and realized compilation charges, so the comparison does not attribute differences in compiler quality to the decision rule. Each table ratio divides a policy’s mean cost by PACE’s mean cost in the same condition. Aggregate results are the equally weighted means of these ratios. Reported percentage savings are the equally weighted means of $100(1 - K_{\text{PACE}}/K_{\text{baseline}})$ across conditions, using each condition’s mean costs. We use 2000 paired repetition-bootstrap draws, conditional on the tested conditions and supplied profiles. These are development comparisons rather than independent confirmation after algorithm selection. The main tables give point estimates, with complete conditional intervals in Appendix C.7.

5.1 SAVINGS

Across the grids, ReAct costs 1.251 and 1.289 times PACE on average (Table 3). No saved prefix exceeds the implemented 1.25 bound over 4800 grid runs and 120 log runs. This checks the implementation on those runs. Theorem 4.1 covers all histories allowed by the model. AutoRPA’s largest grid ratios to PACE are 14.085 and 19.269, while ToolPro stays near ReAct cost by allowing no reuse beyond one task in its compilation criterion.

Table 3: Cost ratios relative to PACE over two grids of 300 conditions each. Mean is the mean of the ratios for individual conditions, and max is the largest tested condition mean. Blue, amber, and green identify GLM, DeepSeek, and Qwen, respectively. Bold marks the best displayed values. Conditional 95% intervals are in Appendix C.7.

Policy	Grid 1 mean ($\times$ PACE) $\downarrow$			Grid 1 max ($\times$ PACE) $\downarrow$			Grid 2 mean ($\times$ PACE) $\downarrow$			Grid 2 max ($\times$ PACE) $\downarrow$		
ReAct	1.528	1.155	1.069	4.543	4.070	2.232	1.585	1.190	1.090	5.487	3.247	2.979
AutoRPA	1.542	2.226	2.375	4.234	13.875	14.085	1.601	2.296	2.471	5.178	18.652	19.269
ToolPro	1.506	1.155	1.069	4.431	4.070	2.232	1.554	1.190	1.090	5.198	3.247	2.979
PACE	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000

On the recorded logs, PACE reduces token costs by 21.85% relative to ReAct, with a conditional 95% interval of [19.66, 24.15]% (Table 4). It saves cost in ten of twelve conditions. Its largest cost ratio to

Table 4: Cost ratios relative to PACE on four recorded arrival sequences, using ten paired repetitions. Blue, amber, and green identify GLM, DeepSeek, and Qwen, respectively. Bold marks the best displayed values. Conditional 95% intervals are in Appendix C.7.

Policy	Sepsis ($\times$ PACE) $\downarrow$			BPI 2019 ($\times$ PACE) $\downarrow$			Wiki tools ($\times$ PACE) $\downarrow$			Wiki humans ($\times$ PACE) $\downarrow$		
ReAct	0.972	1.011	0.993	2.236	1.571	1.480	2.511	1.510	1.381	1.112	1.056	1.039
AutoRPA	0.984	1.170	1.219	2.198	1.628	1.559	2.462	1.542	1.416	1.095	1.168	1.197
ToolPro	0.968	1.011	0.993	2.154	1.571	1.480	2.482	1.510	1.381	1.105	1.056	1.039
PACE	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000

ReAct is 1.029, and its lowest is 0.398. Relative to PACE, AutoRPA and ToolPro have mean cost ratios of 1.470 and 1.396, respectively, although they cost less than PACE in one and two conditions. The comparison concerns the stated decision-rule adaptations, rather than the original systems end to end.

5.2 COMPONENT EFFECTS

Table 5 reports cost ratios relative to full PACE. *No savings test* compiles whenever eligible while retaining the budget. *No cost budget* removes reservations and budget-triggered manifest clearing. *Eager retry* removes both, and *Equal attempt costs* substitutes $C/\hat{p}$ for $\hat{B}$ while retaining actual charges and reservations.

Table 5: Cost ratios after removing or simplifying PACE components. Values are mean cost ratios relative to PACE, which is exactly one. Blue, amber, and green identify GLM, DeepSeek, and Qwen, respectively. Paired intervals are in Appendix C.8. Grid max is the largest tested condition-mean ratio, without a sampling interval.

Variant	Grid 1 mean ($\times$ PACE)			Grid 2 mean ($\times$ PACE)			Logs mean ($\times$ PACE)			Grid max ($\times$ PACE)		
PACE	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
No savings test	1.104	1.185	1.204	1.106	1.144	1.166	1.327	1.568	1.499	1.881	1.859	1.788
No cost budget	1.028	1.015	1.007	1.028	1.016	1.012	1.000	1.000	1.000	1.472	1.397	1.302
Equal attempt costs	1.008	1.115	1.083	1.007	1.088	1.071	1.000	1.312	1.088	1.402	1.585	1.319
Eager retry	2.321	11.756	12.650	2.385	13.322	14.353	1.395	4.487	3.930	14.813	111.240	151.251

Removing the savings test raises paired mean log cost by 46.46% (conditional 95% interval [41.60, 51.74]%). Ignoring failed compilation costs raises it by 13.36% ([12.05, 15.04]%). The GLM log effect of equal costs is exactly zero because those profiles already set $C^{\text{fail}} = C$. Both variants keep the budget, so their largest grid ratios to ReAct remain near the same 1.25 limit despite higher mean costs. Removing the budget changes mean log cost by only -0.014% , yet raises the largest tested grid ratio to ReAct from 1.2494 to 1.8387, or 47.2%. Thus the near-zero log effect concerns average cost, while the budget’s measured contribution is protection against larger losses. Appendix C.8 gives the cost ratios and paired intervals, including conditions where an ablation is cheaper.

Cost assumptions. Varying the missing failed/successful compilation cost ratio over 0.5, 1, 2, 5 gives PACE mean log ratios from 0.788 to 0.830. Assigning high compilation costs to frequent families with low verification probability raises the mean to 0.997, nearly removing the saving while preserving the cost constraint. This assignment makes the families with the most reuse opportunities costly to acquire. The resulting loss of savings shows why task frequency alone is insufficient to select compilations. The budget still limits extra spending, but it cannot create savings when the available compilation costs and useful lifetimes make reuse unprofitable. Appendix C.5 retains all seventy-two sensitivity conditions. Appendix D reports a separate live GUI component study.

6 RELATED WORK

Experience reuse includes executable skill libraries in Voyager (Wang et al., 2023), extracted trajectory insights in ExpeL (Zhao et al., 2023), and procedural-skill refinement in Skill-Pro (Mi et al., 2026). LATM separates tool making from tool use to amortize creation cost across requests (Cai et al., 2023). PACE studies compilation costs, including unsuccessful attempts, and the online decision of when to compile. Prediction-guided rent-or-buy algorithms study purchase timing under uncertain future demand (Purohit et al., 2018; Gollapudi & Panigrahi, 2019), while conservative contextual bandits constrain cumulative performance relative to a baseline (Kazerouni et al., 2016). PACE uses the latter type of constraint for token charges, with measured compilation profiles and reservations that remain valid when the reuse proposal is wrong.

7 LIMITATIONS

The measurements cover seven parameterized task families and three model profiles. Broader studies could include workflows that span several applications, longer procedures, and task descriptions with greater variation. We measure model-token cost. Adding latency, device use, and program-maintenance effort would support decisions that must satisfy several resource constraints. Model and provider updates can change the measured costs, so periodic remeasurement would help assess how long a profile remains useful. Experiments across longer deployment periods could also measure how maintenance effort grows as applications change. Finally, the current implementation receives task-family labels from the workload. A family-discovery component whose token use is included in routing costs would extend the implementation to unlabeled task streams.

8 CONCLUSION

Compiling a computer-use task can make later runs cheap but requires substantial initial spending whose amount is uncertain. PACE measures both costs and decides when to compile under a budget relative to the cost of running every task with the agent. With valid action-cost bounds, the budget limits cost after every arrival, including failed compilations and streams that end before payback. In the experiments using recorded arrival orders, PACE reduces token costs by 21.9% relative to ReAct on average, while ablations separate the effects of compilation selection, failed compilation costs, and the cost budget.

AI USE STATEMENT

We used AI tools for literature retrieval, drafting and revising text, and implementing the measurement and evaluation methods. The tools also assisted with experiment planning, aggregation and interpretation of recorded results, developing and checking mathematical arguments, and preparation of tables and figures. AI-generated programs are also part of the compilation method evaluated in Section 3. The authors are responsible for checking the sources, results, mathematical claims, and final manuscript.

ETHICS STATEMENT

The study uses benchmark applications, an Android emulator, desktop applications in a Linux container, and locally hosted web applications. The arrival experiments use public event logs. The study does not involve recruiting human participants or executing generated actions on users’ personal accounts. Our cost bound does not ensure task correctness, so applying the method to consequential actions would require checks appropriate to those actions.

REPRODUCIBILITY STATEMENT

The code and retained records are at <https://anonymous.4open.science/r/computer-use>. Appendix A describes the task families, recorded model identifiers, cost calibrations, and verification procedures. Section 4.3 states and proves the cost bound, and Appendix B gives the complete execution order and reference calculations. Appendix C specifies the experimental grids, arrival logs, cost substitutions, paired outcomes, and confidence interval calculation. Appendix D describes the live GUI study and its charge records. The artifact includes measurement readers, versioned experiment entry points, paired results, table renderers, and manifests identifying their inputs.

REFERENCES

- Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. *arXiv preprint arXiv:2305.17126*, 2023. URL <https://arxiv.org/abs/2305.17126>.
- Minghao Chen, Xinyi Hu, Zhou Yu, and Yufei Yin. AutoRPA: Efficient GUI automation through LLM-driven code synthesis from interactions. *arXiv preprint arXiv:2605.21082*, 2026a. ICML 2026.
- Sen Chen, Tong Zhao, Yi Bin, Fei Ma, Wenqi Shao, and Zheng Wang. D-GARA: A dynamic benchmarking framework for GUI agent robustness in real-world anomalies. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2026b. arXiv:2511.16590.
- Salma El Yadouni and Guanyi Li. Tracecompiler: Skill-guided mining and compilation of LLM agent traces into mostly deterministic workflows. *arXiv preprint arXiv:2608.02680*, 2026.
- Sreenivas Gollapudi and Debmalaya Panigrahi. Online algorithms for rent-or-buy with expert advice. In *International Conference on Machine Learning (ICML)*, pp. 2319–2327, 2019.
- Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, and Jie Tang. CogAgent: A visual language model for GUI agents. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. arXiv:2312.08914.
- Abbas Kazerouni, Mohammad Ghavamzadeh, Yasin Abbasi-Yadkori, and Benjamin Van Roy. Conservative contextual linear bandits. *arXiv preprint arXiv:1611.06426*, 2016. URL <https://arxiv.org/abs/1611.06426>.
- Juyong Lee, Taywon Min, Minyong An, Dongyoon Hahm, Haeone Lee, Changyeon Kim, and Kimin Lee. Benchmarking mobile device control agents across diverse configurations. In *Conference on Lifelong Learning Agents (CoLLAs)*, 2025. arXiv:2404.16660.
- Bojie Li. PreAct: Computer-using agents that get faster on repeated tasks. *arXiv preprint arXiv:2606.17929*, 2026.
- Mugeng Liu, Shuoqi Li, Yixuan Zhang, and Yun Ma. Beyond static endpoints: Tool programs as an interface for flexible agentic web services. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2606.19992.
- Felix Mannhardt. Sepsis cases – event log. 4TU.ResearchData, 2016. doi:10.4121/uuid:915d2bfb-7e84-49ad-a286-dc35f063a460.
- Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. Skill-Pro: Learning reusable skills from experience via non-parametric PPO for LLM agents. In *International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=9kJQjx2B80>.
- Manish Purohit, Zoya Svitkina, and Ravi Kumar. Improving online algorithms via ML predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- Christopher Rawles, Sarah Clinckemaillie, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyi Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. AndroidWorld: A dynamic benchmarking environment for autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2405.14573.
- Geert Trooskens, Aaron Karlsberg, Anmol Sharma, Lamara De Brouwer, Max Van Puyvelde, Matthew Young, John Thickstun, Gil Alterovitz, and Walter A. De Brouwer. Compiled AI: Deterministic code generation for LLM-based workflow automation. *arXiv preprint arXiv:2604.05150*, 2026.

- Karen Ullrich, Jingtong Su, Claudia Shi, Arjun Subramonian, Amir Bar, Ivan Evtimov, Nikolaos Tsilivis, Randall Balestriero, Julia Kempe, and Mark Ibrahim. Openapps: Simulating environment variations to measure UI-agent reliability. In *International Conference on Learning Representations (ICLR)*, 2026. URL <https://openreview.net/forum?id=cj1MAx7lKs>. arXiv:2511.20766.
- Boudewijn van Dongen. BPI challenge 2019. 4TU.ResearchData, 2019. doi:10.4121/uuid:d06aff4b-79f0-45e6-8ec8-e19730c248f1.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- Zora Zhiruo Wang, Apurva Gandhi, Graham Neubig, and Daniel Fried. Inducing programmatic skills for agentic tasks. In *Conference on Language Modeling (COLM)*, 2025a. arXiv:2504.06821.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In *International Conference on Machine Learning (ICML)*, 2025b. arXiv:2409.07429.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Duling Xu, Zheng Chen, Zaifeng Pan, Jiawei Guan, Dong Dong, Jialin Li, and Bangzheng Pu. Skillsmith: Compiling agent skills into boundary-guided runtime interfaces. *arXiv preprint arXiv:2605.15215*, 2026.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.
- Chaoyun Zhang, Liqun Li, Shilin He, Xu Zhang, Bo Qiao, Si Qin, Minghua Ma, Yu Kang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. UFO: A UI-focused agent for Windows OS interaction. In *Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL)*, 2025. arXiv:2402.07939.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. *arXiv preprint arXiv:2308.10144*, 2023. URL <https://arxiv.org/abs/2308.10144>.
- Longtao Zheng, Rundong Wang, Xinrun Wang, and Bo An. Synapse: Trajectory-as-exemplar prompting with memory for computer control. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2306.07863.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024.

A MEASUREMENT DETAILS

A.1 TASK FAMILIES

The study covers seven parameterized task families across Android, desktop, and web interfaces. Four come from AndroidWorld (Rawles et al., 2025): adding a contact, deleting a Markor note, adding a calendar event, and saving an OsmAnd marker. The desktop tasks save a small Calc table or a Writer memo in a Linux container based on the OSWorld environment (Xie et al., 2024). The web task posts a comment on WebArena’s self-hosted Reddit application (Zhou et al., 2024). The desktop and web tasks use custom parameterized procedures and task-specific state checks. They extend the interface coverage of the measurement without constituting evaluations of the full OSWorld or WebArena benchmarks.

The recorded model identifiers are `z-ai/glm-5.3-flash`, `deepseek/deepseek-v4-flash-vision-exp`, and `qwen/qwen3.8-flash`. The tables abbreviate these profiles as GLM, DeepSeek, and Qwen. Each agent reads an unmarked screenshot and a numbered text tree at every step. The tree is an accessibility tree on Android and desktop, and a DOM tree on the web. The planned comparison has 21 family-model cells. Qwen’s provider refuses the web request before producing a compilation record, and Qwen’s Calc program prevents the replay from completing within its deadline. We report the retained partial Calc spend and distinguish both cases from an ordinary verification rejection.

A.2 COST ACCOUNTING

Input, cached-input, and output tokens can have different prices. For a call with x input tokens, including x_{cache} cached tokens, and y output tokens, its weighted token cost is $(x - x_{\text{cache}}) + r_c x_{\text{cache}} + r_o y$. Here r_c and r_o are the cached-input and output prices relative to the uncached-input price. One unit therefore costs as much as one uncached input token for that model. We sum these units over serving, compilation, and repair calls. The cost calculation excludes wall-clock time, device resources, and human effort. Dollar comparisons across models also require their uncached-input prices.

We subtract the measured no-task harness baseline from agent runs. We do not subtract it from compilation or binding extraction. The Android baselines are 5090 weighted tokens for GLM, 2290 for DeepSeek, and about 1032 for Qwen. Qwen’s calibration includes mostly cache hits, so this value is smaller than its raw-token baseline. The desktop and web calculations use their own recorded calibrations.

We calculate costs from retained call records. Some earlier Android exploration records need estimated cache counts, and their deployment costs use the recorded bill divided by the model’s uncached-input price. Desktop and web costs use recorded token counts, including retained retries and repair calls. Calls lost during interrupted desktop runs cannot be recovered from these records.

A.3 COMPILATION

Three completed agent traces provide the source experience for each initial compilation attempt. A translator rewrites their actions, and a builder constructs the parameterized program, following AutoRPA’s translation and construction stages (Chen et al., 2026a). The model extracts one binding from each source task prompt using the program’s parameter schema. We recover the prompts from the source records or reconstruct them from the recorded task seeds with the same task generator. We permit one retry when the extracted output fails the JSON or type check and charge both calls when a retry occurs. We cache the three extracted bindings within the compilation attempt and use them for every candidate with the same parameter interface. An extraction failure rejects the attempt without substituting the benchmark’s known parameter values. The verifier restores each source task’s initial state and admits a program only if it completes all three tasks. The checkers inspect application records or saved file contents to score the state delivered by the program.

The verifier stops each candidate check at the first failed task and records subsequent tasks as untested. The repair loop uses the failed execution to analyze the error, complete the task with the agent, and refine the program. We allow at most three refinements for each original task and charge each

refinement to the task that first failed the candidate check. After refinement, we check the three tasks again from the beginning and stop repair once all three pass. When an isolated browser worker cannot retain the failed execution’s interface state, the agent restarts the original task and we record the restart and its full charge. This validation reuses construction tasks, while the thirty subsequent deployment tasks provide a separate evaluation.

We reuse archived source traces, translations, and initial three-trace programs with their recorded charges. We run verification from each initial program and record extraction and repair calls. We reuse a historical deployment record only when the verifier selects a byte-identical program under the same deployment input and execution protocol. Otherwise, we repeat deployment for the selected program.

The compilation cost includes translation, the selected builder call, verification-time binding extraction, and every retained model call used to analyze failures, continue an agent run for repair, or refine the program. It excludes the source agent runs because they are accounted for on the agent path. It also excludes the one- and two-trace comparison builders from the selected three-trace cost. Executing the program and checking its result make no model calls after binding extraction. An unsuccessful attempt retains all of its translation, construction, and repair charges.

After verification, we deploy each verified program on thirty fresh bindings. Both verification and deployment use model-extracted bindings, but deployment uses fresh task prompts and a new extraction on each task. The deployment failure fraction therefore includes failures in the complete extraction-and-execution path.

A.4 INITIAL SERVING PROFILES

Table 6 contains the initial exploration and deployment profiles used in the experiments. These differ from the matched replay means in the main text.

Table 6: Serving costs and estimated savings for the initial programs. Both cost columns use thousands of the weighted tokens defined in Section 2. c is the mean cost of baseline-subtracted agent exploration runs, while c^{prog} and q summarize thirty program uses. The saving share charges one mean-cost agent fallback for a failed use. Blue, amber, and green identify GLM, DeepSeek, and Qwen, respectively. Appendix A.8 reports confidence intervals for the deployment estimates. A dash denotes an unavailable program-path value, and the Qwen web request was refused by the provider.

Family	c (K tokens)			c^{prog} (K tokens)			q (fraction)			Saving share (fraction)		
Contacts	98.7	65.0	39.2	0.179	0.430	0.547	0.00	0.03	0.03	1.00	0.96	0.95
Markor delete	33.7	10.8	21.5	0.200	0.402	0.384	0.20	0.03	0.00	0.79	0.93	0.98
Calendar	314	146	135	0.558	–	0.836	0.00	–	0.00	1.00	–	0.99
OsmAnd marker	280	153	122	0.475	–	–	0.00	–	–	1.00	–	–
Calc table	505	647	169	0.672	–	–	0.00	–	–	1.00	–	–
Writer memo	70.2	50.7	37.0	0.547	0.650	0.775	0.00	0.00	0.00	0.99	0.99	0.98
Reddit comment	124	40.0	–	0.651	0.654	–	0.00	0.00	–	0.99	0.98	–

For the trace-count diagnostic, we retain the archived unrepaid programs and their five generated evaluation inputs, supplied directly to each program. This separate diagnostic compares construction choices, while the three-source-task verifier determines admission. Among nineteen cells with trace-count comparisons, the unrepaid three-trace program passes fewer bindings than the best one- or two-trace program in six cells, more in four, and the same number in nine. Five of six verified desktop and web programs require repair, with Qwen Writer the exception.

A.5 REPEATED COMPILATION

We repeat compilation to check whether an initial success or failure persists. We make two additional attempts for each of the four Android families on GLM and DeepSeek, and for the three initially verified Android families on Qwen. This gives twenty-two additional attempts and three observations for each family–model combination tested. Fourteen additional attempts produce a verified program,

five fail verification, and three end after repeated empty provider replies. We count the provider failures as unsuccessful end-to-end attempts and distinguish them from failed program checks.

Table 7: Repeated compilation and checks of the original programs. Verified counts successful compilations out of three, including the initial attempt. Each † marks an attempt lost to repeated empty provider replies. Supplied and extracted columns rerun the original artifact on five new bindings with known or model-extracted parameters. Blue, amber, and green identify GLM, DeepSeek, and Qwen, respectively. The two rejected DeepSeek artifacts were also checked, while Qwen OsmAnd has no additional compilation attempts. The three compilation attempts share traces and translation, so the verified count is not assigned an independent-trial confidence interval.

Family	Verified (of 3)			Supplied (of 5)			Extracted (of 5)		
Contacts	3/3	3/3	3/3	4/5	3/5	5/5	5/5	5/5	5/5
Markor delete	3/3	3/3	3/3	5/5	5/5	5/5	3/5	2/5	5/5
Calendar	2/3†	0/3††	2/3	4/5	0/5	4/5	4/5	0/5	4/5
OsmAnd marker	1/3	0/3	—	1/5	0/5	—	4/5	0/5	—

Contacts and Markor verify on all three attempts for each model. DeepSeek Calendar and OsmAnd verify on none of their three attempts. GLM Calendar and Qwen Calendar verify twice, and GLM OsmAnd verifies only on the initial attempt. The initial outcome therefore persists for some cells and changes for others. The repetitions use a different emulator setup from the original run and include only three observations per cell. They do not establish independent retry outcomes or a stable within-family verification probability. We report these mixed initial and repeated attempts as counts, without a confidence interval for a stationary verification probability.

The call records also let us separate the selected three-trace builder and repair charges from the additional comparison builders. Repeated builds reuse the initial translations, so this comparison excludes translation costs from both sides. For Qwen Calendar, one successful additional attempt costs 207.1k weighted tokens and one failed additional attempt costs 173.5k, both without repair under the same verifier. For GLM OsmAnd, the initial verified build costs 897.3k before translation, while two failed additional attempts cost 1836.3k and 2104.3k. That GLM comparison shares an accounting boundary but changes the verification procedure. These examples show that failure can cost either less or more than success. They do not identify both conditional mean costs for the remaining families, whose missing-cost assumptions are tested in Section C.5.

A.6 PROGRAM RE-EVALUATION

We also rerun the original programs on new bindings, once with supplied parameters and once with model-extracted parameters. GLM and DeepSeek Markor each pass five of five supplied bindings but only three and two extracted bindings, respectively. GLM OsmAnd changes in the other direction, from one supplied-binding pass to four extracted-binding passes. All extracted values pass the type check. These results show that type validity and initial verification do not ensure success on the full deployment path. Because the two checks were not repeatedly run under identical conditions, their difference does not isolate extraction error.

A.7 PROGRAM FAILURES UNDER GUI CHANGES

We test existing programs in a clean environment and under nine GUI perturbations adapted from two robustness benchmarks (Lee et al., 2025; Chen et al., 2026b). The probe executes programs without model calls and measures the fraction of clean-passing bindings that fail under each perturbation. The five configuration changes use font scales of 1.3 and 0.85, display density 0.85 times the default, French locale, and dark theme. The four interruptions are a notification, low battery, a permission dialog, and an update prompt implemented as a notification-based stand-in. Device fingerprints check that each change is reverted before the next condition. Based on the mean retained-success ratios across tested families, appearance changes give failure fractions of 0.060, 0.100, and 0.067 on GLM, DeepSeek, and Qwen, respectively. Interruptions give corresponding fractions of 0.592, 1.000, and 0.917. Across all probe failures, including failures in the clean condition, detectable execution errors account for about 93, 83, and 77 percent on the three models. The remaining failures reach an incorrect task state without an execution error, which limits error-triggered fallback. The perturbation

failure rates describe the selected changes and tested families. They do not estimate how frequently a deployment encounters those changes, so the online evaluation treats the drift hazard as a supplied scenario parameter.

A.8 MEASUREMENT CONFIDENCE INTERVALS

Cost intervals resample the recorded binding indices, retaining the association between agent cost, extraction cost, and program outcome. They condition on the recorded program and binding protocol. Success and failure intervals use an independent Bernoulli working model. Zero failures in thirty trials gives a two-sided 95% interval of $[0, 0.116]$, so a narrow saving-share bootstrap cannot rule out unseen failure modes. The adaptive exploration runs, single initial compilation bills, and three mixed initial/repeated attempts do not support independent-trial cost intervals. The matched replays use a later emulator profile and therefore do not isolate model randomness.

Table 8: Confidence intervals for the thirty-use deployment estimates for each verified initial program. c^{prog} uses weighted tokens and gives the mean followed by positive and negative offsets of its 95 percent binding-bootstrap interval. Failures shows an observed count followed by the lower and upper endpoints of a 95 percent Clopper–Pearson interval under an independent Bernoulli working model. Blue, amber, and green identify GLM, DeepSeek, and Qwen, respectively. These intervals condition on the recorded program and task-binding protocol. The adapted exploration episodes and single acquisition bills do not support an independent-trial cost interval.

Family	c^{prog} (weighted tokens)			Failures (of 30) 95% CI (fraction)		
Contacts	178.7	430.3	546.5	0/30	1/30	1/30
	+15.6	+55.1	+40.9	[0.000,	[0.001,	[0.001,
	−15.0	−42.8	−35.0	0.116]	0.172]	0.172]
Markor delete	200.0	402.1	384.2	6/30	1/30	0/30
	+25.1	+39.3	+25.9	[0.077,	[0.001,	[0.000,
	−26.6	−36.3	−23.6	0.386]	0.172]	0.116]
Calendar	558.4		835.7	0/30		0/30
	+78.8		+41.5	[0.000,		[0.000,
	−75.8		−44.8	0.116]		0.116]
OsmAnd marker	474.8			0/30		
	+45.6			[0.000,		
	−46.1			0.116]		
Calc table	671.9			0/30		
	+32.4			[0.000,		
	−34.6			0.116]		
Writer memo	546.7	649.5	774.7	0/30	0/30	0/30
	+39.0	+38.7	+61.4	[0.000,	[0.000,	[0.000,
	−32.4	−39.1	−59.1	0.116]	0.116]	0.116]
Reddit comment	651.0	654.1		0/30	0/30	
	+75.6	+151.3		[0.000,	[0.000,	
	−67.3	−99.0		0.116]	0.116]	

Table 9: Agent-cost estimates and confidence intervals for the matched replays in Table 1. Each populated entry is the mean cost across thirty recorded bindings. The offsets below the mean give its 95 percent binding-bootstrap interval in K weighted tokens. CV divides the sample standard deviation by the mean. Blue, amber, and green identify GLM, DeepSeek, and Qwen, respectively.

Family	Mean c (K tokens)			Agent CV (ratio)		
Contacts	67.1	67.3	42.2			
	+7.8	+28.2	+1.5	0.326	1.063	0.099
	−7.8	−18.1	−1.4			
Markor delete	35.4	22.3	19.3			
	+4.8	+9.4	+1.0	0.373	1.042	0.128
	−4.5	−7.1	−.8			
Calendar	535.3		315.6			
	+94.4		+55.7	0.485		0.504
	−89.2		−55.1			
OsmAnd marker	258.9					
	+63.1			0.657		
	−60.4					

Table 10: Confidence intervals for success rates and saving shares on the matched replay bindings. Successes gives observed counts and 95 percent Clopper–Pearson endpoints under an independent Bernoulli working model. Saving share gives the mean with positive and negative 95 percent bootstrap offsets, resampling matched agent costs, program charges, and failure indicators together. Blue, amber, and green identify GLM, DeepSeek, and Qwen, respectively. The saving bootstrap cannot reveal a failure mode absent from the sample. An offset marked $< .001$ is positive but smaller than 0.001. For zero failures in thirty uses, the binomial failure interval is $[0, 0.116]$ even when the empirical saving interval is narrow.

Family	Agent successes (of 30)			Program successes (of 30)			Saving share		
	95% CI (fraction)			95% CI (fraction)			(fraction)		
Contacts	30/30 [0.884, 1.000]	27/30 [0.735, 0.979]	30/30 [0.884, 1.000]	30/30 [0.884, 1.000]	29/30 [0.828, 0.999]	29/30 [0.828, 0.999]	0.997 +<.001 -<.001	0.960 +0.035 -0.068	0.954 +0.034 -0.067
Markor delete	30/30 [0.884, 1.000]	23/30 [0.577, 0.901]	30/30 [0.884, 1.000]	24/30 [0.614, 0.923]	29/30 [0.828, 0.999]	30/30 [0.884, 1.000]	0.794 +0.133 -0.166	0.949 +0.038 -0.070	0.980 +0.002 -0.002
Calendar	14/30 [0.283, 0.657]	15/30 [0.313, 0.687]	30/30 [0.884, 1.000]	30/30 [0.884, 1.000]	30/30 [0.884, 1.000]	30/30 [0.884, 1.000]	0.999 +<.001 -<.001	0.997 +<.001 -<.001	0.997 +<.001 -<.001
OsmAnd marker	12/30 [0.227, 0.594]	—	30/30 [0.884, 1.000]	—	—	—	0.998 +<.001 -<.001	—	—

Table 11: New-binding checks of the original artifacts, with 95 percent Clopper–Pearson intervals. Each cell shows the observed count and then the lower and upper endpoints under an independent Bernoulli working model. Blue, amber, and green identify GLM, DeepSeek, and Qwen, respectively. Five checks provide limited evidence about future binding reliability. The candidate-selection and repair gates use their bindings adaptively and are reported as exact counts without test-set intervals.

Family	Supplied (of 5)			Extracted (of 5)		
	95% CI (fraction)			95% CI (fraction)		
Contacts	4/5 [0.284, 0.995]	3/5 [0.147, 0.947]	5/5 [0.478, 1.000]	5/5 [0.478, 1.000]	5/5 [0.478, 1.000]	5/5 [0.478, 1.000]
Markor delete	5/5 [0.478, 1.000]	5/5 [0.478, 1.000]	5/5 [0.478, 1.000]	3/5 [0.147, 0.947]	2/5 [0.053, 0.853]	5/5 [0.478, 1.000]
Calendar	4/5 [0.284, 0.995]	0/5 [0.000, 0.522]	4/5 [0.284, 0.995]	4/5 [0.284, 0.995]	0/5 [0.000, 0.522]	4/5 [0.284, 0.995]
OsmAnd marker	1/5 [0.005, 0.716]	0/5 [0.000, 0.522]	—	4/5 [0.284, 0.995]	0/5 [0.000, 0.522]	—

B ONLINE ALGORITHM DETAILS

Algorithm 1 states the execution order, including trace eligibility, manifest control, fallback, and both compilation outcomes.

B.1 COMPILATION SPEND

For independent attempts with fixed verification probability $p > 0$, the number of failures before a verified program has mean $(1 - p)/p$. The expected compilation spend is therefore

$$B(p) = C + \left(\frac{1}{p} - 1\right) C^{\text{fail}}. \quad (9)$$

Substituting the observed-history estimate $\hat{p}$ gives PACE’s estimated compilation cost $\hat{B}$. This calculation excludes agent service between attempts and assumes a fixed retry distribution, which the repeated GUI measurements do not establish. The cost guarantee instead counts each actual model charge and does not use this independence assumption.

Algorithm 1 PACE with extra-cost margin ϵ . Family identities, cost profiles, routing charges, and the scenario hazard are supplied.

```

1: Initialize cumulative charges  $K \leftarrow 0$  and agent reference  $A \leftarrow 0$ 
2: for each arrival  $t$  do
3:   Load the arriving family's costs and state
4:    $A \leftarrow A + \tau_0 + c$ 
5:   Expire idle entries and tentatively relist the family's retained live program
6:   Compute  $U_t^{\text{serv}}$  from Equation 6
7:   if  $K + U_t^{\text{serv}} > (1 + \epsilon)A$  then
8:     Clear the manifest and select agent service
9:   end if
10:  Pay routing and serve the task; on a detected program failure, pay one agent fallback
11:  Add the routing and serving charges to  $K$ ; log completed agent traces
12:  Invalidate the program after a detected break and update the task arrival records
13:  if at least  $k$  agent runs are complete, no live program exists, and Equation 5 holds then
14:    if  $K + \max(C, C^{\text{fail}}) \leq (1 + \epsilon)A$  then
15:      Compile a candidate and verify it on completed source tasks; increment  $j$ 
16:      if verification passes then
17:         $K \leftarrow K + C$ ; increment  $g$ ; retain and list the program
18:      else
19:         $K \leftarrow K + C^{\text{fail}}$ 
20:      end if
21:    end if
22:  end if
23: end for

```

B.2 PROGRAM LIFETIME

A replacement calculation explains the lifetime cap $1/h$. Suppose a working program can be obtained at deterministic cost B and every use independently breaks it with probability $h > 0$. A replacement cycle has expected length $1/h$, including its terminal break. It pays B , extraction on every use, ordinary fallback with probability q on nonterminal uses, and one terminal fallback. Dividing expected cycle cost by expected length gives

$$c^{\text{prog}} + (1 - h)qc + h(B + c). \quad (10)$$

This is below agent cost c exactly when $s_h/h > B$. At $\hat{H} = 1/h$ with no routing charge, the proposal has the same form, using estimated spend $\hat{B}$. Smaller future-use estimates further limit the uses available to repay compilation. This reference calculation compares two specified policies and does not establish optimal replacement under unknown parameters.

B.3 COST OF A CONSERVATIVE RESERVATION

A single-family example has 300 arrivals, agent cost 1, successful compilation cost 2, possible failed cost 200, and no program-use cost or drift. The compiler succeeds, but the policy does not know that in advance. With $\epsilon = 0.25$, PACE's extra budget never reaches the 200 reservation, so it uses the agent and pays 300. Eager retry pays 3 for source runs and 2 for compilation. The example shows how reserving the failure branch can prevent a profitable compilation while preserving the cost constraint relative to agent execution.

C ONLINE EVALUATION DETAILS

C.1 COST PROFILES

We construct cost profiles for the online evaluation from measurements of seven families for GLM and DeepSeek and six retained families for Qwen. The refused Qwen web request provides no cost profile. A verified cell supplies its agent cost, extraction cost, program failure fraction, and successful compilation cost. A cell without a verified program uses the same model's verified-cell medians

for the unobserved successful-program quantities. The agent success probability is the fraction of completed exploration attempts that succeeded, including unsuccessful attempts in the denominator.

A complete failed compilation supplies that cell’s failed-attempt charge. For cells without such a record, DeepSeek uses its median cost of completed failed compilation attempts and Qwen uses the cost of its one completed failed compilation attempt. The partial Qwen Calc spend is used only for that cell and is excluded from the other failure-cost substitutions. The initial GLM profiles have no complete failed attempt, so their base setting uses $C^{\text{fail}} = C$. These values define fixed charges in the experiments. The missing-cost substitutions are assumptions about unobserved outcomes, not additional measurements.

C.2 SYNTHETIC TASK STREAMS

We evaluate two grids of 300 conditions each. Each crosses three model profiles, four verification settings, five arrival patterns, and five cost or environment settings. The first grid has 1200 arrivals and 30 families that appear repeatedly, except in its sparse pattern. Its verification probabilities are 0.9, 0.5, 0, or an independently assigned per-family mixture over $\{0, 0.1, 0.5, 0.9, 1\}$. Its patterns are uniform family sampling, Zipf sampling, bursts, a change in the frequent family, and infrequent repeats. The uniform pattern concerns family labels, not wall-clock Poisson arrivals. The base setting uses $h = 0.02$, an idle limit of $T = 100$, and router cost $368 + 91\ell_t$. The other settings multiply only failed-compilation cost by five, multiply both compilation costs by five, set $h = 0.1$, or raise the per-entry routing charge to 1820.

The second grid changes both the random seeds and the parameters. It has 1800 arrivals and 40 families that appear repeatedly, except in its sparse pattern, with verification probabilities 0.95, 0.35, 0, or a mixture over $\{0, 0.05, 0.3, 0.7, 1\}$. Its altered-cost settings multiply failed cost by three or both compilation costs by seven, and its drift and routing settings use $h = 0.07$ and $m = 910$. We retain every condition in both grids. The probabilities are assigned independently of the profiles’ initial verification outcomes, so a missing-value substitution does not identify a family’s verification probability.

C.3 RECORDED ARRIVAL ORDERS

We also use Sepsis and BPI 2019 process logs (Mannhardt, 2016; van Dongen, 2019) and two Wikipedia edit streams. In each process log, an arrival is a completed case, ordered by its earliest event, and a family is its full ordered activity sequence. Sepsis contains 1050 arrivals and 846 families, while BPI 2019 contains 251,734 arrivals and 11,973 families. These activity-sequence labels are assigned after each case completes and do not test online family identification.

For Wikipedia, an arrival is an edit, and families group normalized edit-summary templates with bot and tool labels. The tool stream contains 65,564 arrivals and 2529 families. We use the first 100,000 timestamp-ordered human edits, which contain 68,773 families. Human edits with empty or section-only summaries receive separate singleton labels.

The logs supply task repetitions and their order rather than live GUI tasks. For each repetition, measured cost profiles are shuffled onto log-family identifiers, and verification probabilities are assigned independently from the first grid’s mixture. Three models and four logs give twelve combinations of models and logs. The runs use the same base drift and routing settings as the first grid.

Confidence intervals. Each grid condition has eight paired repetitions, and each combination of a model and a log has ten. Compilation outcomes are indexed by family and attempt, while serving outcomes are indexed by family, arrival, and failure channel. Rules in a repetition share these outcome sequences and the same cost mapping. A cell’s cost ratio is the policy’s mean token cost divided by the PACE mean in that cell. An aggregate is the equally weighted mean of cell ratios, without pooling absolute cost units across models. The grid and log intervals use 2000 paired bootstrap resamples, with repetition indices shared across policies and conditions in each study. They describe variation in the retained mappings and outcomes, conditional on the tested scenarios and supplied cost profiles. They do not account for measurement error in the supplied costs or the effects of choosing different

missing-cost substitutions or protocols. The largest tested condition mean is a descriptive maximum and has no confidence interval.

The grids were run during algorithm development, and the second grid changed settings after the first study. The two literature adaptations and the equal-attempt-cost ablation were specified later and evaluated on these same conditions without changing PACE. We preserve both grids, the alternative rules, the separate development cases, and the boundary cases in the result records. The reported comparisons are exploratory evidence rather than independent confirmation of the final algorithm choice. All reported main conditions use detected failures and one agent fallback. The experimental environment reports whether a failure is a persistent break, so invalidation uses a supplied outcome rather than a measured online classifier. Because a failed program uses the same agent outcome as the ReAct baseline on that arrival, nondecreasing task success in this model follows from its construction. The cost bound itself makes no such quality claim.

C.4 COMPARISON RULES

We compare PACE with ReAct, two decision strategies adapted from published systems, and alternative timing rules with and without the same cumulative budget. All methods use the same measured compiler and serving profiles, so the comparison isolates their compilation and reuse decisions. The named adaptations do not reproduce the original systems’ models, code generators, or benchmark scores.

ReAct. The ReAct baseline uses agent execution on every arrival and never compiles or reuses programs (Yao et al., 2023).

AutoRPA (one build). AutoRPA constructs a program from building tasks, limits its repair attempts, and uses ReAct when construction fails (Chen et al., 2026a). Our adaptation attempts one build after $k = 3$ completed agent runs, with the measured compilation charge already including bounded repair. A failed build leaves the family on the agent path. A verified program serves later tasks with agent fallback, and a detected break returns the family to agent service without another build. This transfers the bounded construction-and-reuse policy to the shared compiler and arrival model.

ToolPro (cost adaptation). ToolPro compares construction latency with the latency saved within one task (Liu et al., 2026). We adapt this single-task criterion to token cost: an eligible family requests compilation only when $s_h > \hat{B}$. The estimate $\hat{B}$ uses the same observed verification outcomes as PACE. This rule gives no credit for savings on multiple future tasks. We retain the shared parameterized programs and serving order, so the comparison tests the cost criterion rather than ToolPro’s original tool execution system.

Cost-aware timing controls. The remaining controls share a per-family allowance that admits compilation only when $F + C^{\text{fail}} \leq \max(0, S) + \max(0, \hat{H}s_h)$, where F is failed-compilation spend and S is accumulated program-use saving. The allowance uses estimated future savings and does not enforce PACE’s cumulative bound. *Eager + allowance* requests a build whenever the family is eligible and has no live program. *After 10 arrivals* waits for ten arrivals of the family. *After 10 successes* waits for ten successful agent runs since the last admission or detected break. *Savings threshold* waits until the forgone expected savings on agent execution without program reuse reach $\hat{B}$. *Estimate + allowance* uses PACE’s estimated-savings test with this allowance in place of its cost budget.

The same budget with other proposals. The budgeted rows in Tables 13 and 14 hold PACE’s 0.25 margin, service reservations, and manifest controls fixed. *Eager + both checks*, *Arrival-10 + both*, and *Estimate + both* add that budget to the corresponding estimate-based allowance rules. *Count + budget* changes only PACE’s estimated future uses to $\min(N, 1/h)$. *Optimistic + budget* retains its smoothed count but substitutes the one-sided 90 percent Clopper–Pearson upper value for $\hat{p}$, using it as an optimistic proposal estimate. *One initial try* adds a limit to that optimistic rule: an unsuccessful first build ends compilation for the family, while an initially successful family can later attempt replacement. The budget guarantee does not depend on the calibration of any of these proposal estimates.

All methods require $k = 3$ completed agent runs before compilation and serve the current arrival before attempting a build. They share free artifact lookup, idle expiry, relisting of live programs, and the same failure outcomes. PACE and the other budgeted rules check service and compilation costs, and can clear the manifest before a routing charge. PACE’s main setting is $\epsilon = 0.25$ throughout both comparison tables. Section 5.2 separately removes each of its main components.

C.5 COST-PROFILE SENSITIVITY

The base profiles contain fifteen observed successful compilation costs, four costs from completed failed compilation attempts, and one partial spend. We test the unobserved counterpart by setting nominal ratios $r = C^{\text{fail}}/C$ to 0.5, 1, 2, or 5, while preserving each observed branch cost. For a verified profile we vary C^{fail} , and for a profile with a complete failed attempt we vary C . For Qwen Calc, we retain the imputed C and set the failed cost to the larger of its partial spend and rC . Its actual ratio therefore exceeds the nominal value in the first three settings. The other profile values and the original probability assignments remain fixed. These four settings give forty-eight combinations of models and logs.

We add two dependence scenarios using the original costs. Verification probability decreases from 0.9 to 0.1 as the profile’s rank in C/c increases. One scenario keeps the original profile assignments. The other assigns profiles with high C/c to frequent families while preserving the multiset of assigned profiles. Only the environment constructor uses the full-stream counts, and the policy receives neither those counts nor the hidden verification probabilities. We fixed these seventy-two conditions before evaluation and retained them all. They test specified alternatives to the base assumptions rather than calibrating a deployment’s joint distribution.

Table 12: Sensitivity to missing costs and assigned cost dependencies. Each row uses twelve combinations of models and logs and ten paired repetitions. The first three columns show mean cost ratios to PACE for each model, and the last combines all models. Nonzero-width 95 percent paired bootstrap intervals appear as upper and lower offsets below the mean; exact zero-width intervals are omitted. Blue, amber, and green identify GLM, DeepSeek, and Qwen, respectively. The listed $r = C^{\text{fail}}/C$ is nominal because Qwen Calc’s retained partial-spend floor can raise its actual ratio. Both policies use the same $\epsilon = 0.25$ budget.

Scenario r : cost ratio	ReAct ($\times$ PACE)			PACE ($\times$ PACE)			Equal attempt costs ($\times$ PACE)			Equal / PACE (all models)
$r = 0.5$	1.829	1.158	1.256	1.000	1.000	1.000	1.001	0.998	1.043	1.014
	+ .334	+ .068	+ .106				+ .008	+ .011	+ .018	+ .007
	− .236	− .053	− .069				− .005	− .006	− .016	− .007
$r = 1$	1.708	1.237	1.242	1.000	1.000	1.000	1.000	1.000	1.013	1.004
	+ .265	+ .106	+ .120						+ .006	+ .002
	− .193	− .078	− .077						− .006	− .002
$r = 2$	1.579	1.277	1.237	1.000	1.000	1.000	1.037	1.043	1.023	1.034
	+ .215	+ .134	+ .122				+ .010	+ .014	+ .011	+ .005
	− .144	− .092	− .077				− .008	− .013	− .010	− .005
$r = 5$	1.364	1.286	1.134	1.000	1.000	1.000	1.157	1.169	1.102	1.143
	+ .195	+ .152	+ .049				+ .042	+ .027	+ .047	+ .032
	− .101	− .101	− .033				− .027	− .021	− .051	− .028
Cost-dependent p	2.299	2.129	1.401	1.000	1.000	1.000	1.186	1.049	1.078	1.078
	+ .260	+ .288	+ .165				+ .070	+ .024	+ .023	+ .023
	− .205	− .261	− .112				− .056	− .019	− .019	− .019
Cost-dependent p + repeats	1.010			1.000	1.000	1.000	1.095	1.064	1.053	1.053
	+ .008	1.000	1.000				+ .002	+ .002	+ .001	+ .001
	− .007						− .002	− .002	− .001	− .001

Across the four missing-cost settings, PACE’s mean cost ranges from 0.788 to 0.830 of ReAct. The Equal attempt costs variant costs 1.034 and 1.143 times PACE on average at $r = 2$ and $r = 5$. The comparison also has losses: PACE costs more in four of twelve conditions at $r = 0.5$ and in one at $r = 5$. At $r = 1$, the eight GLM and DeepSeek conditions tie exactly, while the Qwen partial-spend floor leaves differences in the other four. The value of separate costs therefore depends on the assigned asymmetry and its effect on compilation timing.

Under cost-dependent verification alone, PACE has a mean cost of 0.657 times ReAct cost. Adding the assignment of costly profiles to frequent families raises that ratio to 0.997, so the base saving nearly disappears. Equal attempt costs has a mean cost ratio of 1.050 in the same joint scenario. Both

variants satisfy the implemented budget in all 1440 protected runs. These results separate the cost constraint from the amount saved, which depends on the workload and supplied cost model.

C.6 ALTERNATIVE COMPILATION PROPOSALS

Applying PACE’s budget to the cost-aware timing controls preserves their low costs. On the recorded logs, Eager + both checks, Arrival-10 + both, and Estimate + both have mean costs of 0.7824, 0.7809, and 0.7817 times ReAct cost, compared with 0.7815 for PACE. Each paired 95 percent interval for control cost divided by PACE cost includes one. These comparisons do not establish a cost advantage for PACE’s proposal over the protected cost-aware controls.

Count + budget has mean cost ratios of 0.9090 and 0.9066 on the two grids, below PACE’s 0.9095 and 0.9070. Its log mean is 0.7819, and its paired ratio to PACE has interval $[0.9999, 1.0008]$. The count rule has a similar cost to the smoothed estimate in these conditions. Optimistic + budget and One initial try have higher log means of 0.8512 and 0.8473. Together, these results support cost-aware proposals within the shared cost budget, but do not identify the exact use-count estimate as a separate source of improvement.

C.7 COMPLETE COST COMPARISONS

The following tables retain the named adaptations and all constructed compilation rules, with conditional paired bootstrap intervals. PACE’s ratio is one in every repetition by normalization. ToolPro matches ReAct in the DeepSeek and Qwen conditions shown, so those rows have identical ratios and intervals relative to PACE. We omit offsets only for zero-width intervals, not for small nonzero intervals. Maxima are descriptive maxima over tested condition means and have no sampling interval.

C.8 COMPONENT COSTS

Table 15 gives cost ratios relative to PACE for the ablations, and Table 16 gives their paired percentage changes with intervals. A change is the mean of $100(K_{\text{variant}}/K_{\text{PACE}} - 1)$ over fixed combinations of models and conditions, using each condition’s mean costs. Effects can interact and are not additive shares of a total saving. PACE’s zero change is exact by construction. On the recorded logs, equal attempt costs changes no GLM proposal because its base profiles already have $C^{\text{fail}} = C$. The mean log effect of removing the budget is small, while its worst grid cost increases. By contrast, the savings-test and cost-estimate ablations retain the same budget, which keeps their maximum cost ratios to ReAct near the same limit. Equal attempt costs is cheaper than PACE in 41 and 48 conditions of the two grids, so its average loss is not uniform across conditions.

Table 13: Cost across the two parameter grids, relative to PACE. Each grid contains one hundred conditions per model and eight paired repetitions per condition. Mean is the mean of the cost ratios for individual conditions, and Max is the largest tested condition mean. Nonzero-width 95 percent paired bootstrap intervals appear as upper and lower offsets below the mean; exact zero-width intervals are omitted. Intervals use 2000 shared repetition resamples and condition on the tested scenarios and supplied profiles. An offset marked $< .001$ is positive but smaller than 0.001. Blue, amber, and green identify GLM, DeepSeek, and Qwen, respectively. AutoRPA and ToolPro are decision-rule adaptations defined in Appendix C.4. The selected comparisons appear in Tables 3 and 4. PACE is exactly one by normalization; other fixed ratios also have zero-width intervals. The descriptive maxima have no sampling interval.

Policy	Grid 1 mean ($\times$ PACE) $\downarrow$			Grid 1 max ($\times$ PACE) $\downarrow$			Grid 2 mean ($\times$ PACE) $\downarrow$			Grid 2 max ($\times$ PACE) $\downarrow$		
ReAct	1.528 +.051 -.041	1.155 +.035 -.030	1.069 +.018 -.014	4.543	4.070	2.232	1.585 +.072 -.053	1.190 +.033 -.028	1.090 +.013 -.008	5.487	3.247	2.979
AutoRPA	1.542 +.047 -.035	2.226 +.055 -.026	2.375 +.031 -.024	4.234	13.875	14.085	1.601 +.055 -.042	2.296 +.064 -.039	2.471 +.023 -.017	5.178	18.652	19.269
ToolPro	1.506 +.050 -.042	1.155 +.035 -.030	1.069 +.018 -.014	4.431	4.070	2.232	1.554 +.070 -.054	1.190 +.033 -.028	1.090 +.013 -.008	5.198	3.247	2.979
Eager + allowance	1.056 +.004 -.004	1.023 +.007 -.005	1.022 +.003 -.003	1.507	1.408	1.389	1.062 +.003 -.003	1.018 +.004 -.002	1.038 +.009 -.006	1.504	1.407	1.419
After 10 arrivals	1.053 +.003 -.003	1.025 +.007 -.005	1.015 +.003 -.003	1.473	1.408	1.389	1.067 +.005 -.005	1.020 +.003 -.002	1.031 +.009 -.006	1.503	1.407	1.419
After 10 successes	1.191 +.014 -.013	1.078 +.012 -.011	1.042 +.006 -.004	2.402	1.525	1.275	1.214 +.012 -.011	1.084 +.012 -.010	1.064 +.009 -.008	2.282	1.570	1.456
Savings threshold	1.091 +.008 -.007	1.042 +.008 -.006	1.027 +.004 -.003	1.476	1.397	1.296	1.098 +.004 -.003	1.046 +.003 -.002	1.038 +.008 -.006	1.481	1.401	1.310
Estimate + allowance	1.028 +.002 -.002	1.022 +.007 -.005	1.008 +.001 -.001	1.469	1.386	1.284	1.029 +.002 -.002	1.016 +.002 -.001	1.012 +.004 -.003	1.472	1.397	1.302
Eager + both checks	1.017 +.003 -.003	1.004 +.007 -.004	1.012 +.003 -.003	1.162	1.111	1.076	1.021 +.003 -.003	1.002 +.003 -.002	1.021 +.008 -.006	1.189	1.191	1.217
Arrival-10 + both	1.021 +.003 -.002	1.005 +.007 -.004	1.004 +.003 -.002	1.393	1.111	1.060	1.033 +.005 -.004	1.004 +.003 -.002	1.013 +.008 -.006	1.391	1.191	1.217
Estimate + both	1.000 +<.001 -<.001	1.005 +.007 -.004	1.001 +.002 -.001	1.029	1.228	1.047	1.002 +.002 -.002	1.000 +<.001 -<.001	1.001 +.003 -.002	1.069	1.024	1.067
Count + budget	0.997 +<.001 -<.001	0.999 +<.001 -<.001	1.000 +<.001 -<.001	1.034	1.006	1.018	0.997 +<.001 -<.001	0.999 +<.001 -<.001	1.000 +<.001 -<.001	1.035	1.004	1.008
Optimistic + budget	1.053 +.007 -.006	1.123 +.016 -.010	1.115 +.010 -.007	1.409	1.423	1.298	1.052 +.007 -.006	1.087 +.012 -.008	1.099 +.006 -.005	1.296	1.460	1.350
One initial try	1.135 +.056 -.053	1.125 +.018 -.016	1.103 +.012 -.011	2.752	1.581	1.572	1.103 +.032 -.025	1.084 +.025 -.019	1.078 +.008 -.007	2.480	1.706	1.343
PACE	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000

Table 14: Cost on four recorded arrival sequences, relative to PACE. Each value divides the policy’s mean cost by PACE’s mean cost in the same condition, using ten paired repetitions. Nonzero-width 95 percent paired bootstrap intervals appear as upper and lower offsets below the mean; exact zero-width intervals are omitted. Intervals use 2000 shared repetition resamples and condition on the tested scenarios and supplied profiles. An offset marked $< .001$ is positive but smaller than 0.001. Blue, amber, and green identify GLM, DeepSeek, and Qwen, respectively. AutoRPA and ToolPro are decision-rule adaptations defined in Appendix C.4. The selected comparisons appear in Tables 3 and 4. PACE is exactly one by normalization; other fixed ratios also have zero-width intervals.

Policy	Sepsis ($\times$ PACE) $\downarrow$			BPI 2019 ($\times$ PACE) $\downarrow$			Wiki tools ($\times$ PACE) $\downarrow$			Wiki humans ($\times$ PACE) $\downarrow$		
ReAct	0.972 +.020 -.021	1.011 +.057 -.058	0.993 +.013 -.014	2.236 +.454 -.336	1.571 +.289 -.217	1.480 +.220 -.150	2.511 +.968 -.644	1.510 +.562 -.296	1.381 +.346 -.209	1.112 +.017 -.016	1.056 +.017 -.015	1.039 +.013 -.012
AutoRPA	0.984 +.021 -.022	1.170 +.048 -.051	1.219 +.033 -.030	2.198 +.451 -.334	1.628 +.296 -.222	1.559 +.231 -.152	2.462 +.937 -.623	1.542 +.569 -.300	1.416 +.355 -.215	1.095 +.016 -.015	1.168 +.018 -.016	1.197 +.015 -.013
ToolPro	0.968 +.019 -.020	1.011 +.057 -.058	0.993 +.013 -.014	2.154 +.434 -.319	1.571 +.289 -.217	1.480 +.220 -.150	2.482 +.963 -.641	1.510 +.562 -.296	1.381 +.346 -.209	1.105 +.017 -.016	1.056 +.017 -.015	1.039 +.013 -.012
Eager + allowance	1.014 +.004 -.004	1.003 +.005 -.003	1.004 +.006 -.005	1.003 +.005 -.006	1.001 +.002 -.003	1.008 +.035 -.036	1.000 +.015 -.018	1.001 +.008 -.008	0.947 +.041 -.049	1.011 +.002 -.001	1.001 +.001 -.001	1.008 +.001 -.002
After 10 arrivals	0.992 +.009 -.009	1.006 +.004 -.004	1.007 +.007 -.007	1.006 +.005 -.005	1.002 +.002 -.003	1.008 +.035 -.036	1.004 +.015 -.017	1.001 +.008 -.008	0.948 +.041 -.049	1.006 +.001 -.001	1.001 +.001 -.001	1.007 +.001 -.002
After 10 successes	0.989 +.009 -.009	1.012 +.019 -.016	1.012 +.006 -.007	1.250 +.083 -.064	1.138 +.052 -.046	1.124 +.063 -.046	1.416 +.362 -.209	1.148 +.130 -.078	1.036 +.077 -.061	1.032 +.005 -.005	1.018 +.002 -.002	1.020 +.005 -.004
Savings threshold	1.001 +.002 -.002	1.001 +.003 -.002	1.007 +.005 -.005	1.103 +.036 -.031	1.045 +.009 -.009	1.075 +.027 -.029	1.121 +.074 -.055	1.064 +.069 -.040	1.011 +.049 -.043	1.011 +.002 -.002	1.007 +.001 -.002	1.008 +.002 -.002
Estimate + allowance	1.000	1.000	1.000	1.000 +.001 -<.001	1.000 +.002 -.002	1.020 +.031 -.022	1.004 +.009 -.005	0.996 +.004 -.006	1.000 +.001 -.001	1.000 +.001 -<.001	1.000 +.001 -<.001	0.999 +.001 -<.001
Eager + both checks	1.014 +.004 -.004	1.003 +.005 -.003	1.004 +.006 -.005	1.003 +.005 -.006	1.001 +.002 -.003	1.009 +.035 -.036	1.001 +.015 -.018	1.001 +.008 -.008	0.947 +.041 -.049	1.011 +.002 -.001	1.001 +.001 -.001	1.008 +.001 -.002
Arrival-10 + both	0.992 +.009 -.009	1.006 +.004 -.004	1.007 +.007 -.007	1.006 +.005 -.005	1.002 +.002 -.003	1.009 +.035 -.036	1.004 +.015 -.017	1.002 +.008 -.008	0.948 +.041 -.049	1.006 +.001 -.001	1.001 +.001 -.001	1.007 +.001 -.002
Estimate + both	1.000	1.000	1.000	1.000 +.001 -<.001	1.000 +.002 -.002	1.004 +.011 -.005	1.004 +.009 -.004	0.998 +.004 -.006	1.000 +.001 -.001	1.000 +.001 -<.001	1.000 +.001 -<.001	0.999 +.001 -<.001
Count + budget	1.000 +.001 -.001	1.000	1.007 +.005 -.005	1.000 +.001 -<.001	1.000 +.001 -<.001	1.000 +.001 -<.001	0.999 +.001 -<.001	0.999 +.001 -<.001	0.999 +.001 -<.001	1.000 +.001 -<.001	1.000 +.001 -<.001	1.000 +.001 -<.001
Optimistic + budget	1.039 +.014 -.014	1.140 +.042 -.042	1.120 +.038 -.037	1.034 +.009 -.008	1.110 +.019 -.019	1.050 +.042 -.047	1.032 +.020 -.013	1.085 +.028 -.022	0.995 +.046 -.053	1.039 +.002 -.002	1.187 +.009 -.009	1.147 +.006 -.005
One initial try	0.977 +.019 -.020	1.080 +.047 -.048	1.038 +.022 -.020	1.383 +.237 -.196	1.059 +.045 -.042	1.116 +.127 -.082	1.411 +.360 -.278	1.107 +.141 -.104	1.066 +.198 -.110	1.017 +.008 -.006	1.063 +.007 -.007	1.053 +.006 -.006
PACE	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000

Table 15: Cost ratios for the component ablations, relative to PACE. The first three columns show mean condition-wise cost ratios to PACE, with positive and negative offsets of the 95 percent paired bootstrap interval. Grid max is the largest tested condition mean across both grids and all three models. Blue, amber, and green identify GLM, DeepSeek, and Qwen, respectively. An offset marked $< .001$ is positive but smaller than 0.001. Equal attempt costs assumes $C^{\text{fail}} = C$ when estimating compilation cost, while keeping actual charges and reservations unchanged. No cost budget removes the protection layer, including manifest control.

Variant	Grid 1 mean ($\times$ PACE)			Grid 2 mean ($\times$ PACE)			Logs mean ($\times$ PACE)			Grid max ($\times$ PACE)
PACE	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
No savings test	1.104	1.185	1.204	1.106	1.144	1.166	1.327	1.568	1.499	1.881
	+0.11	+0.32	+0.16	+0.11	+0.13	+0.10	+0.46	+1.24	+1.22	
	-.008	-.018	-.010	-.010	-.009	-.006	-.052	-.092	-.078	
No cost budget	1.028	1.015	1.007	1.028	1.016	1.012	1.000	1.000	1.000	1.472
	+0.02	+0.02	<.001	+0.02	+0.02	+0.01	<.001	<.001	<.001	
	-.002	-.002	<.001	-.001	-.001	-.001	<.001	<.001	<.001	
Equal attempt costs	1.008	1.115	1.083	1.007	1.088	1.071		1.312	1.088	1.585
	+0.03	+0.18	+0.07	+0.03	+0.10	+0.06	1.000	+0.36	+0.28	
	-.003	-.011	-.006	-.003	-.008	-.005		-.026	-.026	
Eager retry	2.321	11.756	12.650	2.385	13.322	14.353	1.395	4.487	3.930	151.251
	+0.69	+9.22	+4.19	+0.50	+7.48	+6.07	+0.96	+8.28	+6.21	
	-.082	-.336	-.201	-.055	-.331	-.367	-.106	-.841	-.501	

Table 16: Cost change after removing or simplifying PACE components. Δ is the mean percentage cost change from full PACE across conditions. Positive values mean higher cost. Blue, amber, and green identify GLM, DeepSeek, and Qwen, respectively. Offsets show the conditional 95 percent paired interval in percentage points; exact zero-width intervals are omitted. Grid max is the largest tested condition-mean ratio, without a sampling interval.

Variant	Grid 1 Δ cost (%)			Grid 2 Δ cost (%)			Logs Δ cost (%)			Grid max ($\times$ PACE)		
PACE	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.000	1.000	1.000
No savings test	10.4	18.5	20.4	10.6	14.4	16.6	32.7	56.8	49.9	1.881	1.859	1.788
	+1.1	+3.2	+1.6	+1.1	+1.3	+1.0	+4.6	+12.4	+12.2			
	-.8	-1.8	-1.0	-1.0	-.9	-.6	-5.2	-9.2	-7.8			
No cost budget	2.8	1.5	0.7	2.8	1.6	1.2	-0.005	-0.030	-0.006	1.472	1.397	1.302
	+2	+2	<.1	+2	+2	+1	+0.05	+0.17	+0.04			
	-.2	-.2	<.1	-.1	-.1	-.1	-.008	-.023	-.006			
Equal attempt costs	0.8	11.5	8.3	0.7	8.8	7.1		31.2	8.8	1.402	1.585	1.319
	+3	+1.8	+7	+3	+1.0	+6	0.0	+3.6	+2.8			
	-.3	-1.1	-.6	-.3	-.8	-.5		-2.6	-2.6			
Eager retry	132.1	1075.6	1165.0	138.5	1232.2	1335.3	39.5	348.7	293.0	14.813	111.240	151.251
	+6.9	+92.2	+41.9	+5.0	+74.8	+60.7	+9.6	+82.8	+62.1			
	-8.2	-33.6	-20.1	-5.5	-33.1	-36.7	-10.6	-84.1	-50.1			

D LIVE GUI EXECUTION

We test agent fallback and call-cost reservations on the OpenApps Calendar web application (Ulrich et al., 2026). This study uses a fixed program and a scheduled compilation attempt, so it does not evaluate PACE’s online compilation choice.

D.1 PAIRED TASK EXECUTION

Four seeds specify event title, date, description, location, URL, and invitees. Each runs in a clean interface, with a renamed submit button, and with a blocking dialog, giving twelve agent/program pairs with matching initial states. Both paths use GLM 5.3 Flash and the same natural-language goal. The agent reads screenshots and an accessibility tree in headless Chromium. The program receives a model-extracted binding and uses Playwright locators. An execution error triggers agent fallback from the current GUI state, without resetting it. A task has a total limit of twenty model calls. The evaluator checks the created event and preservation of unrelated application state.

Table 17: Observed outcomes and serving bills for twelve paired Calendar tasks. Mean bill bounds are in milli-USD and include extraction and any fallback. Their endpoints combine known charges with the retained reservations for unknown bills; they are not confidence intervals. Billing stops remain unsuccessful outcomes, and the agent has one additional step-limit failure. Serving costs exclude the fixed program’s original compilation and the separate compilation attempt.

Serving path	Success (of 12)	Billing stops (count)	Mean bill bounds (milli-USD)
Agent	7/12	4	[8.142, 21.590]
Program + fallback	11/12	1	[1.202, 4.564]

The agent succeeds in seven of twelve tasks, with four billing interruptions and one step-limit failure. All four clean program runs succeed without fallback. All eight perturbed program runs invoke fallback, and seven succeed after retaining all six target fields at handoff. The remaining run has no correct target fields at handoff and stops on a billing interruption. Only five perturbed pairs expose both policies to the intervention before termination. Both succeed in all five. This post-exposure subset does not replace the twelve-pair comparison. Tables 18 and 19 retain all outcomes, including interrupted and denied tasks.

Table 18: All prespecified paired tasks, including provider interruptions and unsuccessful execution. Bills use milli-USD; brackets are lower and upper accounting bounds, while unbracketed bills are settled. P+F denotes program execution followed by agent fallback on an execution error. Fields counts the six target values already correct when fallback begins. All pairs start from matching application and interface states.

Seed	Condition	Agent	P+F	Agent bill (milli-USD)	P+F bill (milli-USD)	Fields (of 6)
0	Clean	Bill stop	Success	[5.147, 45.493]	0.048	–
0	Renamed submit	Bill stop	Success	[2.050, 42.396]	0.763	6/6
0	Blocking dialog	Step limit	Success	17.156	1.255	6/6
1	Clean	Success	Success	9.687	0.048	–
1	Renamed submit	Success	Success	6.831	0.763	6/6
1	Blocking dialog	Success	Success	11.091	1.129	6/6
2	Clean	Bill stop	Success	[14.067, 54.413]	0.048	–
2	Renamed submit	Success	Success	8.658	0.891	6/6
2	Blocking dialog	Bill stop	Bill stop	[0.000, 40.346]	[7.354, 47.699]	0/6
3	Clean	Success	Success	6.802	0.049	–
3	Renamed submit	Success	Success	7.876	0.893	6/6
3	Blocking dialog	Success	Success	8.334	1.184	6/6

D.2 RESERVATIONS

Before a call, the runner reserves a charge using declared input/output limits and the recorded price schedule. A confirmed bill replaces the reservation. An unsuccessful response or a response without a confirmed bill ends the task without a retry and retains the reserved charge. All 221 physical requests

Table 19: Four tight-budget diagnostics in their prespecified arrival order. Costs and prefix totals use milli-USD. The reference increases by 20 milli-USD per scheduled arrival, including a denied arrival. The prefix total includes charges and retained reservations for unsettled calls. The first task is denied before any model call or GUI service, so this diagnostic does not satisfy the theorem’s affordable-default-action assumption.

Arrival	Condition	Outcome	Bill (milli-USD)	Prefix ceiling (milli-USD)	Prefix total (milli-USD)
1	Renamed submit	Denied	0.000	25.000	0.000
2	Blocking dialog	Success	1.257	50.000	1.257
3	Renamed submit	Success	0.890	75.000	2.147
4	Blocking dialog	Bill stop	[0.215, 40.561]	100.000	42.708

pass the recorded admission arithmetic. Of these, 215 have verified bills and six remain unsettled. Known charges total USD 0.121359, with an upper bound of USD 0.363432 after including unsettled reservations.

The main reference adds USD 0.08 per scheduled task and permits 1.25 times the cumulative reference. A separate attempt, after three completed agent traces, costs USD 0.006872, passes its predeclared two-instance gate, and leaves the fixed serving program unchanged. Its charge is included in the cumulative cost checked against the main budget, but is excluded from the serving-cost comparison. This one-call attempt has no repair and differs from the three-task verification with model-extracted bindings in the main measurement.

Four additional tasks use a separate reference of USD 0.02 per arrival. Two succeed, one stops on a billing interruption, and the first is denied because its USD 0.039834 binding reservation exceeds the initial USD 0.025 allowance. No model call or GUI service occurs for that denied task, although it still increases the cumulative reference by USD 0.02. All recorded prefixes remain within their declared ceilings. These references are supplied allowances, not measured bills from running every task with the agent, and the denial falls outside the theorem’s affordable-default-service premise.

D.3 COST BOUNDS WITH UNSETTLED BILLS

For each request without a confirmed bill, we retain the known charge as a lower bound and the reservation as an upper bound. Across all twelve pairs, mean serving bills are bounded by [8.142, 21.590] milli-USD for the agent and [1.202, 4.564] for the program path. We do not restrict this comparison to the eight pairs with fully settled bills. For a paired difference, the lower endpoint subtracts the agent’s upper bill from the program’s lower bill. The upper endpoint uses the opposite endpoints. The mean difference is bounded by $[-20.388, -3.577]$ milli-USD.

We enumerate all $4^4 = 256$ ordered bootstrap draws of the four seed clusters, retaining both policies and all three conditions within each selected seed. Using linear interpolation, we combine the 2.5 percentile of the lower mean differences and the 97.5 percentile of the upper mean differences. This gives a 95% envelope of $[-33.017, 4.621]$ milli-USD, which includes zero. It propagates billing bounds through the bootstrap, rather than providing an exact-bill confidence interval. The four clusters, one task family, and provider interruptions limit population inference and conclusions about successful GUI service.